컨텍스트, 메시지, 기억과 세션

모델이 보게 되는 세계가 어떻게 선택·정규화·압축·저장되고, 재개 시 다시 유효한 대화로 조립되는지 읽는다

이 권을 마치면 context를 "프롬프트 문자열"이 아니라 출처와 순서, 손실, cache, 재개 규칙을 가진 실행 표현으로 설명할 수 있다.

STUDY SNAPSHOT
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

오늘 우리는 이 코드의 어느 부분을 함께 보는가

사용자의 한 줄 입력은 곧바로 API로 가지 않는다. 명령 문법을 통과하고, 여러 attachment와 memory를 만나며, 내부 message union으로 정규화되고, token 압력 아래 compact되며, session journal에서 복구된다. 이 권은 정보가 많아지는 과정뿐 아니라 무엇이 의도적으로 사라지고 무엇이 끝까지 보존되는지를 추적한다.

58	23,964	161	358
분류된 TS/TSX 파일	물리 코드 줄	async 함수	tool use 표기

CHAPTER 01

사용자 입력은 작은 언어로 해석된다

같은 입력 상자에 들어온 문자열이라도 slash command, shell shortcut, 일반 prompt는 서로 다른 효과와 기록 방식을 가진다.

CHAPTER 02

메시지는 대화 기록이면서 실행 프로토콜이다

role과 text만으로는 tool pairing, partial output, compaction boundary, metadata를 표현할 수 없다. 메시지 모델은 런타임의 내부 IR에 가깝다.

CHAPTER 03

기억은 저장보다 선택이 어렵다

memory 파일이 존재하는 것만으로 모델이 기억하는 것은 아니다. 발견·관련성 판단·prompt 삽입·갱신이 함께 작동해야 한다.

CHAPTER 04

압축과 재개는 과거를 다시 쓰는 작업이다

token 한도를 넘지 않기 위해 과거를 줄이되, 다음 행동을 정당화하는 사실과 미완료 효과는 보존해야 한다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

사용자 입력은 작은 언어로 해석된다

같은 입력 상자에 들어온 문자열이라도 slash command, shell shortcut, 일반 prompt는 서로 다른 효과와 기록 방식을 가진다.

이 장에서 실제로 펼쳐볼 경로

src/utils/processUserInput/** · src/commands/** · src/constants/prompts.ts

processUserInput은 문자열 분기보다 큰 경계다

입력을 어떤 언어로 읽을지 결정하는 순간 권한과 기록 의미가 갈린다.

일반 텍스트는 모델 입력이 되지만 slash command는 로컬 상태를 바꾸거나 별도 화면을 열 수 있고, shell shortcut은 곧바로 외부 효과를 일으킬 수 있다. 같은 TextInput이라도 semantic category가 다르다.

이 경계를 parser로 보면 명령 인식, 인자 해석, 실행을 분리해야 하는 이유가 보인다. 인식과 효과가 한 함수에 붙으면 autocomplete, 도움말, audit가 실제 실행 규칙과 쉽게 어긋난다.

새 입력 문법을 추가할 때는 우선순위와 escaping을 반드시 검토하자. 일반 prompt로 보내려던 문자열이 명령으로 오해되거나, 명령 인자가 transcript에서 다른 형태로 남을 수 있기 때문이다.

스터디에서 확인할 질문

명령 prefix를 escape해 일반 모델 입력으로 보내는 규칙은 어디에서 소유해야 가장 일관적인가?

slash command는 UI 메뉴가 아니라 로컬 capability다

명령 목록에는 모델을 거치지 않고 세션·설정·화면을 바꾸는 기능이 모여 있다.

`src/commands/**`를 보면 각 명령은 이름과 설명만 가지지 않는다. 활성 조건, 인자 처리, React 화면, state update가 결합된 작은 application service처럼 동작한다.

이 구조는 명령을 빠르게 추가하기 좋지만, 명령이 app state 내부 구조를 직접 알기 시작하면 command catalog가 두 번째 composition root가 된다. 특히 session을 지우거나 permission mode를 바꾸는 명령은 화면 편의 기능이 아니라 제어 평면이다.

스터디에서는 명령을 기능별로 외우기보다 effect class로 분류해 보자: pure query, UI navigation, session mutation, external effect. 이 분류는 테스트와 권한 요구를 결정한다.

스터디에서 확인할 질문

새 명령이 "사용자 명시 입력"이라는 이유만으로 별도 permission 확인 없이 외부 효과를 실행해도 되는가?

attachment는 부가 파일이 아니라 context의 typed source다

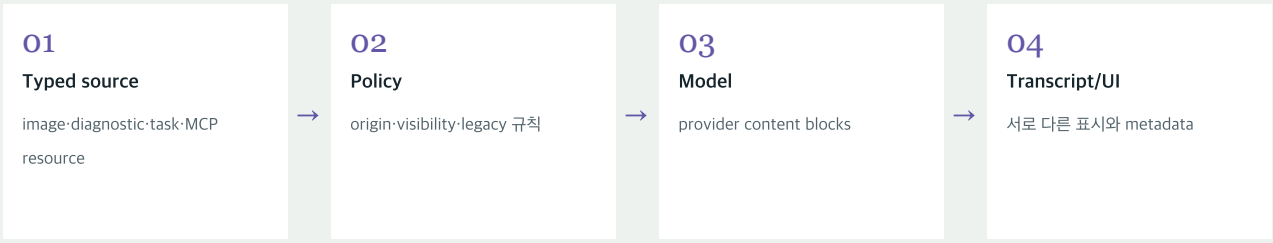
이미지, 진단, MCP resource, task 상태, hook context가 한 message로 낮아지기 전까지 서로 다른 의미를 유지한다.

`normalizeAttachmentForAPI` 주변의 큰 switch는 attachment 종류가 얼마나 넓은지 보여 준다. 각 case는 단순 직렬화가 아니라 숨김 여부, provenance, 표시 문구, legacy 호환을 함께 정한다.

모든 attachment를 일찍 문자열로 바꾸면 구현은 단순해지지만 출처와 보안 등급을 잃는다. 반대로 typed object를 끝까지 유지하면 provider adapter가 복잡해진다. 현재 코드는 내부에서 타입을 유지하고 API 직전에 표현을 낮추는 쪽에 가깝다.

새 attachment를 설계할 때 model-visible content, transcript-visible content, UI-only metadata를 따로 적자. 세 projection이 같을 필요는 없지만 차이는 의도적이어야 한다.

Attachment가 세 projection으로 갈라지는 과정



이 그림이 감추지 않는 것 한 번 문자열로 평탄화하면 source type과 authority를 잃는다. 그래서 내부 union을 유지하다 consumer 경계에서 projection한다.

스터디에서 확인할 질문

hook이 추가한 context와 사용자가 붙인 파일은 모델에는 모두 text여도 감사와 표시에서는 왜 달라야 하는가?

system prompt는 정적 문서가 아니라 구성 결과다

도구, 모델, 작업 디렉터리, 모드, 조직 정책이 서로 다른 조각을 더한다.

QueryEngine은 기본 prompt, memory mechanics, append prompt를 배열로 조립한다. 이때 순서는 단순한 문서 편집 순서가 아니라 지시의 범위와 cache 안정성에 영향을 준다.

프롬프트 조각을 한 거대한 상수로 합치면 최종 문자열은 보기 쉽지만 어떤 capability가 어떤 지시를 요구했는지 provenance가 사라진다. 반대로 조각이 지나치게 분산되면 실제 모델 입력을 재구성하기 어렵다.

좋은 개선은 prompt fragment에 owner, activation condition, intended audience를 붙이고 최종 assembly trace를 관찰 가능하게 만드는 것이다. 단, 사용자 데이터나 비밀 prompt를 로그에 그대로 남기지 않아야 한다.

스터디에서 확인할 질문

같은 지시가 system prompt와 tool description에 중복될 때 어느 쪽을 canonical owner로 둘 것인가?

02

메시지는 대화 기록이면서 실행 프로토콜이다

role과 text만으로는 tool pairing, partial output, compaction boundary, metadata를 표현할 수 없다. 메시지 모델은 런타임의 내부 IR에 가깝다.

이 장에서 실제로 펼쳐볼 경로

src/utils/messages.ts · src/types/** · src/utils/queryContext.ts

discriminated union이 가능한 상태를 제한한다

메시지 type과 subtype은 실행 중 어떤 사건이 존재할 수 있는지 코드로 적는다.

assistant, user, system message는 content만 다른 레코드가 아니다. compact boundary, hook result, task notification처럼 runtime 의미가 있는 system subtype들이 별도 형태를 가진다. TypeScript의 discriminated union은 각 case에서 필요한 필드를 좁혀 준다.

이 메커니즘의 가치는 autocomplete보다 변경 파급을 드러내는 데 있다. 새 subtype을 추가하면 normalization, rendering, persistence가 exhaustive한지 확인할 수 있다. 다만 any와 broad cast가 많아지면 그 안전망은 사라진다.

타입 리팩터링은 먼저 실제 variant 목록과 consumer matrix를 만든 뒤 해야 한다. 누락된 원본 type 파일이 있는 복원 저장소에서는 컴파일 성공만으로 완전성을 주장할 수 없다는 점도 함께 기억하자.

tag가 TypeScript control-flow narrowing을 만드는 방식

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
01  type Message =
02    | { type: "assistant"; blocks: Block[] }
03    | { type: "system"; subtype: Boundary }
04    | { type: "user"; content: Content[] }
05
06  if (message.type === "system") {
07    handleBoundary(message.subtype)
08  }
```

1. type field
   runtime에도 남는 식별자라 persistence와 switch 양쪽에서 쓸 수 있다.
2. narrowing
   분기 안에서 해당 variant의 field만 접근하게 한다.
3. exhaustiveness
   never 검사를 쓰면 새 variant의 누락 consumer를 compile time에 찾을 수 있다.

언어·런타임 안쪽에서는 TypeScript 타입은 실행 시 사라지지만 tag 문자열은 남는다. 그래서 외부 JSON에는 runtime schema가, 내부 분기에는 union narrowing이 모두 필요하다.

스터디에서 확인할 질문

unknown variant를 만났을 때 오래된 세션을 살리기 위해 무시할 것인가, 데이터 손실을 막기 위해 실패할 것인가?

정규화는 정보를 버리는 결정이다

provider가 이해하지 못하는 내부 메타데이터를 제거하면서도 protocol 필드는 지켜야 한다.

Query context 변환은 내부 message에서 API message를 만든다. 이 과정에서 UI용 필드와 내부 origin은 빠질 수 있지만 role 순서, tool_use/tool_result id, content block 순서는 유지되어야 한다.

정규화를 단순 map 함수로 보면 삭제된 정보의 의미를 놓친다. 어떤 필드는 privacy 때문에 보내지 않고, 어떤 필드는 provider schema에 없어서 보내지 않으며, 어떤 필드는 다음 turn 계산에만 필요하다. 같은 "제거"라도 근거가 다르다.

변환 함수 옆에 field-level comment를 늘어놓기보다, source-to-target mapping 테스트와 금지 필드 테스트를 두는 편이 더 강한 계약이 된다.

스터디에서 확인할 질문

내부 provenance를 provider에 보내지 않으면서도 응답을 원래 source와 다시 연결하려면 어떤 correlation data가 필요한가?

tool pairing은 메시지 정리 단계에서도 다시 검사된다

실행 당시 맞았던 짝도 compaction과 resume 뒤에는 고아가 될 수 있다.

messages 유틸리티에는 orphaned tool result 제거, 중복 id 보정, transcript가 mid-turn에서 시작하는 경우의 repair가 있다. 이는 message 배열이 언제나 이상적인 API 대화로 저장된다고 가정하지 않는다는 뜻이다.

repair는 편리하지만 위험하다. 조용히 삭제하면 세션은 살아나도 어떤 효과가 실제로 있었는지 설명이 사라질 수 있다. 따라서 API 호환을 위한 projection repair와 원본 journal 보존을 구분해야 한다.

재개 경로를 고칠 때는 "API가 받아들인다"를 성공 기준으로 삼지 말자. 사용자에게 보이는 과거, 실제 파일 상태, 모델이 보게 될 과거가 얼마나 일치하는지 확인해야 한다.

스터디에서 확인할 질문

고아 tool result를 제거한 사실 자체를 사용자나 telemetry에 알려야 하는 조건은 무엇인가?

메시지 ID는 여러 projection을 연결하는 축이다

화면의 행, transcript record, tool result, SDK event가 같은 사건을 가리키려면 안정된 identity가 필요하다.

코드를 따라가면 uuid, request id, tool_use id, source assistant uuid가 서로 다른 관계를 표현한다. 하나의 만능 id가 아니라 session/turn/message/tool-call 수준이 구분되어 있다.

ID를 단순 문자열로 두면 전달은 쉽지만 잘못된 수준의 id를 섞어도 타입이 막지 못한다. branded type이나 명시적 이름이 유지보수에 도움이 되는 이유다. 실제로 id는 정렬 키가 아니라 인과 관계를 복구하는 열쇠다.

새 projection을 추가할 때 화면 편의를 위해 id를 재생성하지 말고 canonical identity를 전달하자. 필요한 경우 projection-local key와 domain identity를 별도 필드로 둔다.

스터디에서 확인할 질문

재시도된 provider request는 같은 request id를 유지해야 하는가, 같은 turn 아래 새 attempt id를 가져야 하는가?

03

기억은 저장보다 선택이 어렵다

memory 파일이 존재하는 것만으로 모델이 기억하는 것은 아니다. 발견·관련성 판단·prompt 삽입·갱신이 함께 작동해야 한다.

이 장에서 실제로 펼쳐볼 경로

src/memdir/** · src/services/extractMemories/** · src/services/autoDream/**

memory directory는 작은 지식 저장소다

경로와 파일 형식은 저장 위치가 아니라 발견과 범위의 규칙을 만든다.

memdir 코드는 개인·프로젝트·팀 기억의 경로와 scan을 다룬다. 모델은 filesystem 전체를 자동으로 기억하지 않으므로, 어떤 디렉터리를 탐색하고 어떤 파일을 후보로 삼는지가 실제 memory capability를 결정한다.

이 구조는 retrieval system과 닮았지만 vector database와 같지는 않다. 파일 경로, 이름, 최근성, 명시적 prompt가 중요한 신호이며 사용자가 직접 열어볼 수 있다는 장점이 있다.

새 memory 종류를 추가할 때 저장 API보다 먼저 scope와 precedence를 정하자. 전역 기억과 프로젝트 기억이 충돌할 때 어느 것이 우선인지, 오래된 사실을 어떻게 표시할지 없으면 더 많은 memory가 더 나쁜 context를 만든다.

스터디에서 확인할 질문

팀 memory에 개인 선호나 비밀이 들어가지 않도록 쓰기 경계와 읽기 경계를 각각 어디에서 검사할 것인가?

relevance selection이 model world를 결정한다

모든 기억을 넣을 수 없으므로 선택 함수는 곧 정보 손실 정책이다.

`findRelevantMemories`는 현재 작업과 연결된 memory를 찾으려 한다. 여기서 false negative는 필요한 지식을 잃고, false positive는 token과 주의를 낭비한다. 둘의 비용은 대칭이 아니다.

관련성 점수만 보면 provenance와 freshness가 빠진다. 오래된 설정이 높은 유사도로 선택될 수 있고, 팀 규칙보다 개인 메모가 먼저 들어갈 수도 있다. 따라서 selection에는 source, age, scope를 함께 고려할 여지가 있다.

개선 실험은 단순 retrieval hit rate가 아니라 실제 task outcome과 stale-memory harm를 함께 봐야 한다. 현재 코드에서 얻을 수 없는 운영 지표는 보고서가 임의로 성공 기준으로 선언하지 않는다.

스터디에서 확인할 질문

같은 키워드를 가진 상충 memory 두 개를 발견했을 때 모델에 둘 다 보여 줄 것인가, 하나를 선택할 것인가?

extract와 autoDream은 쓰기 권한을 가진 요약기다

대화에서 기억 후보를 만드는 순간 임시 발언이 장기 지식으로 승격될 수 있다.

memory extraction은 세션의 관찰에서 재사용할 사실을 뽑고, autoDream 계열은 유휴 시간에 이를 통합하려 한다. 이 기능은 단순 요약보다 강하다. 다음 세션의 행동을 바꿀 수 있는 durable artifact를 만든다.

따라서 생성 품질뿐 아니라 admission policy가 필요하다. 누가 말했는지, 사실인지 선호인지, 만료 조건은 무엇인지, 사용자 수정은 어떻게 반영되는지를 잃으면 memory가 권위 없는 prompt injection이 된다.

코드를 읽을 때 프롬프트 내용만 보지 말고 write target, secret guard, feature gate, consolidation lock을 함께 보자. 이 주변 장치가 기능의 실제 안전 경계다.

스터디에서 확인할 질문

모델이 추출한 기억을 즉시 쓰는 것과 사용자 검토 후 쓰는 것 사이에서 이 프로젝트가 선택한 기본은 무엇이며 왜 그런가?

기억과 skill은 context에 들어오는 방식이 다르다

기억은 사실 후보이고 skill은 행동 지침이므로 같은 우선순위로 합치면 안 된다.

파일 편집 경로에서 관련 skill directory를 발견하고 비동기로 로드하는 코드는 작업 대상이 capability discovery를 촉발할 수 있음을 보여 준다. 반면 memory는 과거 사실과 선호를 보강한다.

둘 다 text로 내려가더라도 trust semantics는 다르다. skill은 실행 방법을 제시하고 tool 사용을 유도할 수 있어 더 강한 instruction surface다. memory의 우연한 문구를 skill처럼 해석하거나 외부 skill을 개인 기억처럼 신뢰하면 경계가 무너진다.

context assembly에서 source label과 instruction priority를 보존해야 하는 이유가 여기에 있다. 모델에게 보여 주는 최종 문자열보다 그 문자열이 어떤 권위를 갖는지가 중요하다.

스터디에서 확인할 질문

동적으로 발견된 skill이 현재 turn 중간에 추가될 때 이미 만든 prompt cache와 도구 목록은 어떻게 갱신되어야 하는가?

04

압축과 재개는 과거를 다시 쓰는 작업이다

token 한도를 넘지 않기 위해 과거를 줄이되, 다음 행동을 정당화하는 사실과 미완료 효과는 보존해야 한다.

이 장에서 실제로 펼쳐볼 경로

src/services/compact/** · src/utils/sessionStorage.ts · src/history.ts · src/utils/messages.ts

compact는 요약 기능이 아니라 lossy state migration이다

원본 message history가 더 작은 표현으로 바뀌며 다음 turn의 행동 기반도 달라진다.

auto compact는 오래된 대화를 요약하고 boundary message를 남긴다. 이때 사라진 token에는 단순 잡담뿐 아니라 파일을 왜 바꿨는지, 어떤 tool이 실패했는지 같은 결정 근거가 포함될 수 있다.

따라서 좋은 summary는 "내용을 짧게" 하는 것이 아니라 future action에 필요한 invariant를 보존해야 한다. 열린 task, 수정된 파일, 거부된 권한, 미해결 오류는 일반 서술보다 구조화된 상태에 가깝다.

압축 품질을 평가할 때 자연스러운 문장과 token 절감만 보지 말자. 압축 전후 동일한 후속 질문에 필요한 사실이 유지되는지, 중복 효과를 유발하지 않는지를 확인해야 한다.

압축 전후에 달라지는 정보의 성격

정보	그대로 보존	요약 가능	다시 읽기 필요
tool_use id	필수		
결정 이유		조건부	
대용량 Read 결과		요약/tombstone	파일 재읽기
미완료 task	상태·id	진행 문구	runtime reconcile

구분해야 하는 이유 compaction은 reversible compression이 아니다. future action을 위한 최소 상태를 먼저 구조화하고 자연어 요약은 그 위에 둬야 한다.

스터디에서 확인할 질문

현재 대화에서 compact 후에도 반드시 보존되어야 할 다섯 가지 실행 사실을 골라 보라.

함께 펼쳐볼 자료 src/services/compact/compact.ts · src/services/compact/autoCompact.ts · src/services/compact/prompt.ts

microcompact는 결과 내용과 tool identity를 분리한다

큰 tool output을 지워도 무엇을 실행했고 무엇을 얻었는지는 추적되어야 한다.

microCompact는 특정 도구 결과를 대상으로 token을 줄이고 어떤 tool id가 정리되었는지 boundary metadata에 남긴다. 이는 출력 payload와 실행 사실을 같은 것으로 보지 않는 설계다.

여기서 함정은 파일을 읽었다는 사실만 남고 읽은 내용이 사라지는 경우다. 이후 편집이 그 내용에 의존한다면 모델은 stale한 자신감을 가질 수 있다. 반대로 모든 결과를 유지하면 긴 세션이 진행되지 않는다.

도구별 compaction policy가 필요한 이유다. 재생성 비용이 낮은 검색 결과와, 재실행하면 다른 값을 내는 외부 명령 결과는 같은 방식으로 버릴 수 없다.

스터디에서 확인할 질문
어떤 tool result는 요약하고 어떤 것은 다시 읽도록 tombstone만 남겨야 하는지 판단 기준을 만들어 보라.

session storage는 화면 상태가 아니라 재구성 재료를 저장한다

재개 가능한 시스템은 최종 문자열보다 사건의 순서와 identity를 필요로 한다.

Session storage와 history 코드를 읽으면 메시지 직렬화, 세션 식별, append와 load가 만난다. 저장 포맷은 현재 UI component state와 분리되어야 다른 표면에서도 같은 세션을 열 수 있다.

내구성은 파일에 썼다는 사실만으로 충분하지 않다. 부분 write, process crash, 버전 변경, 중복 append를 견뎌야 한다. 복구 코드는 그래서 legacy variant와 orphan repair를 다룬다.

포맷 변경에는 migration만큼 downgrade와 unknown field 보존도 중요하다. 오래된 클라이언트가 새 subtype을 만나도 원본을 손상시키지 않는 전략이 필요하다.

스터디에서 확인할 질문
session file 쓰기 중 crash가 나면 다음 시작에서 완전한 마지막 record를 어떻게 판별할 수 있는가?

resume는 저장 파일을 읽는 일이 아니라 일관된 과거를 만드는 일이다

journal, 실제 workspace, provider protocol이 서로 다른 상태일 수 있다.

재개 시 messages는 compact boundary 이후로 잘릴 수 있고, 고아 tool result가 정리되며, 현재 파일은 세션 종료 당시와 달라졌을 수 있다. 즉 저장된 기록을 그대로 배열에 넣는 것만으로는 유효한 실행 상태가 되지 않는다.

복구 과정은 세 truth를 조정한다: 기록된 intent, 관찰된 effect, 현재 외부 상태. 이 셋이 다르면 시스템은 조용히 하나를 정답으로 고르기보다 사용자에게 불확실성을 드러내야 한다.

구조 개선에서는 raw journal과 model-facing repaired projection을 분리하는 방향이 유용하다. repair가 무엇을 지웠는지 추적할 수 있고, 미래 버전이 원본에서 다시 projection을 만들 수 있기 때문이다.

Resume에서 합쳐야 하는 세 가지 truth

01	→	02	→	03	→	04
Journal		**Workspace**		**Protocol**		**Projection**
기록된 intent와 message order		현재 파일·process·task 상태		provider가 수용할 pairing		repair 사실을 보존한 model history

이 그림이 감추지 않는 것 load는 deserialization이고 resume는 reconciliation이다. 동일하지 않은 세 상태를 조용히 하나로 덮지 않는 것이 중요하다.

스터디에서 확인할 질문

resume 직후 첫 모델 호출 전에 어떤 consistency check를 수행하면 중복 실행 위험을 줄일 수 있는가?

코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

discriminated union

언어가 제공하는 것
공통 tag 값으로 variant를 좁혀 각 분기에서 가능한 field를 제한한다.

이 저장소가 쓰는 방식
assistant/user/system·attachment subtype·compact boundary를 구분한다.

놓치기 쉬운 함정
default branch와 broad cast가 exhaustive 검사를 무력화할 수 있다.

normalization

언어가 제공하는 것
여러 source representation을 정해진 target shape와 순서로 낮춘다.

이 저장소가 쓰는 방식
내부 message와 attachment를 provider payload로 바꾼다.

놓치기 쉬운 함정
삭제한 provenance와 UI metadata는 target에서 되살릴 수 없다.

tagged persistence record

언어가 제공하는 것
version과 kind가 있는 record가 미래 reader의 해석 분기를 만든다.

이 저장소가 쓰는 방식
세션·compact boundary·legacy attachment의 재개 호환을 유지한다.

놓치기 쉬운 함정
unknown variant를 조용히 버리면 durable evidence가 손실된다.

lazy projection

언어가 제공하는 것
canonical source를 유지한 채 consumer 시점에 필요한 표현을 계산한다.

이 저장소가 쓰는 방식
model-visible history, transcript, UI history를 다르게 만든다.

놓치기 쉬운 함정
projection을 다시 canonical state로 저장하면 손실이 누적된다.

memoization/cache

언어가 제공하는 것
입력 identity에 따라 계산 결과를 재사용한다.

이 저장소가 쓰는 방식
memory scan과 prompt prefix 비용을 낮춘다.

놓치기 쉬운 함정
cwd·설정·age가 cache key에 없으면 stale context가 된다.

lossy compaction

언어가 제공하는 것
큰 history를 요약·tombstone·boundary로 치환한다.

이 저장소가 쓰는 방식
token window 안에서 장기 세션을 계속한다.

놓치기 쉬운 함정
미완료 효과와 판단 근거를 자연어 요약만으로 보존하기 어렵다.

스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

시작 경로	먼저 볼 표현	그 다음에 물을 질문
src/utils/processUserInput/processUserInput.ts	입력 종류 판별	문자열이 어느 local capability로 해석되는가
src/utils/processUserInput/processSlashCommand.tsx	command lookup/dispatch	인식과 effect가 분리되어 있는가
src/utils/messages.ts	message/attachment switch	variant 추가가 어느 consumer에 전파되는가
src/utils/queryContext.ts	API projection	provider에 보내지 않는 필드의 근거는 무엇인가
src/memdir/findRelevantMemories.ts	관련성 선택	freshness·scope·conflict를 어떻게 다루는가
src/services/compact/compact.ts	요약 입력과 boundary	후속 행동에 필요한 사실을 보존하는가
src/services/compact/microCompact.ts	tool result 축소	payload와 실행 identity가 어떻게 분리되는가
src/utils/sessionStorage.ts	append/load/version	부분 write와 legacy record를 어떻게 복구하는가

코드 독해 규칙

타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

한 입력이 model context가 되기까지의 정보 장부 만들기

추가된 정보만 적지 말고 각 변환에서 사라진 정보, 바뀐 권위, 남은 identity를 함께 기록한다.

1. 일반 prompt, slash command, shell shortcut 세 입력을 골라 `processUserInput`의 서로 다른 결과를 표로 만든다.
2. attachment 하나를 골라 internal representation, model-visible text, transcript 표시를 비교한다.
3. tool_use와 tool_result 한 쌍을 compact 전후로 추적하고 살아남는 id와 사라지는 payload를 적는다.
4. memory와 skill 한 개씩을 골라 activation condition과 instruction authority 차이를 설명한다.
5. 불완전한 mid-turn transcript를 가정하고 resume repair가 무엇을 제거·추가하는지 검토한다.

> **완료의 기준**
>
> context를 문자열 묶음이 아니라 source → typed representation → projection → compacted projection의 계보로 그릴 수 있으면 된다.

다음 모임에 남길 기록

다음 모임에는 정보 손실이 가장 위험한 변환 하나와, 그 손실을 탐지할 수 있는 테스트를 가져온다.